



REDLINE DAYLIGHT • TASK DOCUMENTATION

Reusable Role: generic_package

A production-style Ansible project: **one generic role** that installs and configures **any Linux package** — nginx, apache2, redis, git — driven entirely by variables. Correct structure, full idempotency, four verification checks, and proof the same role deploys a second package with zero edits.

By **Nabawy**

ENGINEERED & DOCUMENTED

TASK DOC

Build + Verify

Role

generic_package

WSL

Ubuntu 22.04+

Idempotent

run twice ✓

Table of Contents

	The Map — Executive Summary	3		Steps 14–15 — Verify nginx · Prove Reuse	10
01	Task Goal + 5 Objectives	4		Variable Precedence + Role Anatomy	11
02	Tools Used + Setup Check	5		The Ansible Concepts in Play	12
03	Steps 1–4 — Skeleton · cfg · Inventory · Defaults	6		The Expected Result — Proven	13
04	Steps 5–7 — vars · Tasks · Handlers	7		Completion Checklist	14
05	Steps 8–10 — Templates · meta · site.yml	8		Troubleshooting Table	15
06	Steps 11–13 — Check · Dry Run · Real Run	9			

► **TASK [How to use this doc]** read order

[**DEBUG**] **Build first, understand second:** pages 6–10 are the build (15 steps, every file in full). Page 11 explains *why* the variable precedence works. Page 12 maps every Ansible concept to where it's used. Page 13 shows the expected outputs. Pages 14–15 are your acceptance gate and your rescue kit. Everything from the task spec is here — nothing trimmed.



The Map — The Whole Task

One role. Any package. Here's the route — every line links to its page.

The mission is set in [Section 1 — Task Goal & the 5 Objectives](#): a production-style project where a single **generic_package** role installs and configures any Linux package — ending with nginx running, enabled at boot, and serving a styled page at `http://localhost` showing package, host, service and port.

The gear list is in [Section 2 — Tools](#): WSL Ubuntu, Ansible 2.16.3, apt/package, systemd, Jinja2, ansible-galaxy, git and an SSH key — plus the setup-check commands.

Then the build across five pages: [Steps 1–4](#) (folder skeleton, ansible.cfg, inventory.ini, role defaults), [Steps 5–7](#) (vars lock, the four guarded tasks, the two ignore_errors handlers), [Steps 8–10](#) (three Jinja2 templates, meta, site.yml), [Steps 11–13](#) (syntax check, --check --diff dry run, the real run twice), and [Steps 14–15](#) (nginx verification six ways + the reusability proof with redis-server).

Then the understanding: [Variable Precedence + Role Anatomy](#) (why defaults lose to playbook vars), [The Concepts in Play](#) (every Objective-3 concept mapped), and [The Expected Result](#) — what each of the four checks looks like when green. Close with the [Completion Checklist](#) and the [Troubleshooting Table](#). Route clear. ►

KEY IDEA **The unifying idea:** the role never mentions a package name in its own code. nginx, redis, apache2 — all of it arrives through variables. Change the vars, change the deployment. That's what "reusable" actually means.



Task Goal — and the 5 Objectives

Build a **production-style Ansible project** using a single generic, reusable role — **generic_package** — to install and configure any Linux package on target hosts. The project must use correct structure (`ansible.cfg` , `inventory` , `site.yml` , `roles/`), use **roles instead of flat playbook tasks**, and be fully **idempotent, testable, and reusable**.

[**KEY IDEA** · **Expected result**] After `ansible-playbook site.yml` : nginx is **installed, running, enabled at boot**, and serves a styled HTML page at `http://localhost` showing the deployed package, host, service, and port.

► **TASK [The 5 Objectives]** complete = all 5 done

1

Correct project structure

Standard Ansible layout — `ansible.cfg` (inventory, roles_path, become, ssh key), `inventory.ini` (web + local groups), `site.yml` calling the role, legacy flat playbooks kept for comparison, and `roles/generic_package/` with tasks, handlers, 3 templates, defaults, vars, meta.

2

Generic role behavior — driven entirely by variables

8 variables control everything: `package_name` (default nginx) · `service_name` (defaults to package name) · `service_state` (started) · `service_enabled` (true) · `package_state` (present) · `app_port` (80) · `template_enabled` (false) · `custom_index_content` (fallback page text). Same role call + different vars = nginx, apache2, or anything else — **without editing the role**.

3

All Ansible concepts used correctly

Variable priority: defaults (lowest) vs vars (high) vs playbook vars vs `-e` (highest) · **Handlers:** Restart + Reload — once at end of play, only if notified, `ignore_errors` so one bad service doesn't abort · **Templates:** `nginx.conf.j2`, `app.conf.j2` (with `if package_name` logic), `index.html.j2` (styled page) · **Idempotency:** run twice → second run reports `ok`, not `changed`.

4

Deploy proof with nginx

`site.yml` calls the role with `package_name: nginx`, `service_name: nginx`, `app_port: 80`, `template_enabled: false`. Nginx installs, starts, stays enabled — and the custom index lands at `/var/www/html/index.html` plus `/var/www/eslam/index.html` (eslam.com vhost test in WSL).

5

Verified with the full command loop

All four checks green: ① `--syntax-check` passes · ② `--check --diff` dry-run shows expected counts with no real changes · ③ the real run installs and starts everything · ④ `curl http://localhost/` returns the styled page from `index.html.j2`.



Tools Used — and the Setup Check

Nine tools, one lab. Each with its exact role in this task:

Tool	Purpose
WSL (Ubuntu 22.04+)	Linux environment on Windows; the target for localhost testing
Ansible core $\geq$ 2.9 (lab on 2.16.3)	Configuration management / automation engine
apt / package module	Install software on target hosts
systemd / service	Manage service state — started / enabled / restarted / reloaded
Jinja2	Templates for nginx config and index.html
ansible-galaxy	Role scaffolding + metadata validation
git	Version control of every playbook, role, and config
VS Code (optional)	Editor; Remote - WSL extension for editing inside WSL
SSH key (/home/eslam/.ssh/id_ed25519)	Auth to remote hosts — configured in <code>ansible.cfg</code>

► **TASK [Setup check commands]** run these first

```
$ ansible --version
$ python3 --version
$ wsl --version      # from Windows PowerShell
```

[**DEBUG · The environment twist**] This lab is unusual: the project lives on Windows (`C:\Users\eslam\OneDrive\Desktop\ansible`) but runs **inside WSL** (`/mnt/c/Users/eslam/OneDrive/Desktop/ansible`). That crossing is where the most common gotcha in this task comes from — see the Troubleshooting table (the `/mnt/c` permissions warning).

[**KEY IDEA**] The package module (not `apt` directly) in the role's install task is a deliberate genericity choice — it routes to the right backend per distribution, keeping the role portable beyond Debian/Ubuntu.

🔑 **TAKEAWAY** The environment: **WSL Ubuntu as both controller and localhost target** · Ansible $\geq$ 2.9 · the genericity rides on `package` + `service` + Jinja2 · version control via git is part of the completion checklist, not an afterthought.

▶▶ Everything ready — the 15-step build begins: → **Steps 1–4**



Skeleton • ansible.cfg • Inventory • Defaults

1 Create project skeleton and folder tree

```
$ mkdir -p roles/generic_package/{tasks,handlers,templates,defaults,vars,meta}
```

[**DEBUG** • **Paths**] Windows: `C:\Users\eslam\OneDrive\Desktop\ansible` · WSL: `/mnt/c/Users/eslam/OneDrive/Desktop/ansible` · For Linux permissions: copy to `~/ansible` then `chmod 644 ansible.cfg` — this avoids the "world writable directory" warning on NTFS /mnt/c.

2 Write ansible.cfg

```
[defaults]
inventory = inventory.ini
roles_path = roles
host_key_checking = False
retry_files_enabled = False
stdout_callback = yaml
private_key_file = /home/eslam/.ssh/id_ed25519

[privilege_escalation]
become = True
become_method = sudo
become_ask_pass = False
```

3 Write inventory.ini

```
[web]
web1 ansible_host=192.168.1.10 ansible_user=ubuntu
web-server1 ansible_host=98.88.29.132 ansible_user=ubuntu

[local]
localhost ansible_connection=local

[web:vars]
ansible_become=true
```

4 Write role defaults — roles/generic_package/defaults/main.yml

Lowest-priority variables — easy to override:

```
---
package_name: nginx
service_name: "{{ package_name }}"
service_state: started
service_enabled: true
package_state: present
app_port: 80
template_enabled: false
custom_index_content: "<h1>Deployed via generic_package role!</h1><p>Package: {{ package_name }} Host: {{ inventory_hostname
```

[**KEY IDEA**] Look at `service_name: "{{ package_name }}"` — a **default that references another default**. Set `package_name` to `redis-server` and the service name follows automatically. This one line is the genericity engine: 8 defaults, 0 package names hard-coded in role logic.



vars Lock • Tasks • Handlers

5 Write role vars — roles/generic_package/vars/main.yml

High-priority lock — commented out = nothing locked in this lab, so playbook vars win:

```
---
# package_name: nginx
# service_name: nginx
```

[**WATCH OUT**] Uncomment those two lines and **nothing in the playbook can override them** — vars/ beats playbook vars in precedence. That's the lock's power and its danger: use it only when a value must never be tuned.

6 Write role tasks — tasks/main.yml (order matters)

```
---
- name: Update apt cache (Debian/Ubuntu)
  apt:
    update_cache: yes
    cache_valid_time: 3600
  when: ansible_os_family == "Debian"

- name: Install package {{ package_name }}
  package:
    name: "{{ package_name }}"
    state: "{{ package_state }}"
  notify: Restart service

- name: Ensure service {{ service_name }} is {{ service_state }} and enabled={{ service_enabled }}
  service:
    name: "{{ service_name }}"
    state: "{{ service_state }}"
    enabled: "{{ service_enabled }}"
  when: service_name is defined and service_name | length > 0
  ignore_errors: true
```

1 apt cache (Debian only, valid 1h) → **2** install `{{ package_name }}` with `{{ package_state }}`, notify Restart → **3** service task guarded by `when` so non-service packages don't break it. (The template/index tasks from the spec land with Step 8's templates.)

7 Write role handlers — handlers/main.yml

```
---
- name: Restart service
  service:
    name: "{{ service_name }}"
    state: restarted
  ignore_errors: true

- name: Reload service
  service:
    name: "{{ service_name }}"
    state: reloaded
  ignore_errors: true
```

Run once at end of play, only if notified — and `ignore_errors` keeps one bad service from aborting the play.

[**KEY IDEA**] Three safety patterns in 20 lines: the `when` guard makes service management **optional per package** (a CLI tool with no service won't fail the play) · `ignore_errors` on both service touchpoints means **one misbehaving service degrades, never aborts** · and `notify` keeps restarts **change-triggered**, not blind.

Templates • meta • site.yml

8 Write the three templates

app.conf.j2 — nginx-specific config when `package_name == "nginx"`, generic comment otherwise:

```
{% if package_name == "nginx" %}
user www-data;
worker_processes auto;
pid /run/nginx.pid;
events {
    worker_connections {{ worker_connections | default(1024) }};
}
http {
    sendfile on;
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    server {
        listen {{ app_port | default(80) }};
        root /var/www/html;
        index index.html;
        location / {
            try_files $uri $uri/ =404;
        }
    }
}
{% else %}
# Generic config for {{ package_name }}
{% endif %}
```

index.html.j2 — styled landing page showing package name, service name, host, port, and a live **"Service Active & Enabled"** status badge (gradient CSS, info boxes, pulse animation). **nginx.conf.j2** — plain distro-style variant using `worker_connections | default(1024)` and `app_port | default(80)`.

9 Write role metadata — meta/main.yml

```
---
galaxy_info:
  author: ansible
  description: Generic package installer - install any package (nginx, apache2, redis, etc.)
  license: MIT
  min_ansible_version: 2.9
  platforms:
    - name: Ubuntu
      versions: [focal, jammy]
    - name: Debian
      versions: [bullseye, bookworm]
  galaxy_tags: [package, system, generic]

dependencies: []
```

10 Write the main playbook — site.yml

```
---
- name: Install nginx via generic role
  hosts: all
  become: true
  roles:
    - role: generic_package
  vars:
    package_name: nginx
    service_name: nginx
    app_port: 80
    template_enabled: false
```



Check · Dry Run · Real Run

11

Syntax and inventory checks

```
$ ansible-playbook site.yml -i inventory.ini --syntax-check
$ ansible -i inventory.ini all --list-hosts
$ ansible-inventory -i inventory.ini --graph
```

① Playbook parses without errors · ② every host listed · ③ groups render as [web] / [local] trees.

12

Dry run — no changes, show diff

```
$ ansible-playbook site.yml -i inventory.ini --limit localhost --check --diff -v
```

Expected: install/template/index tasks report **changed** (dry), the config template task reports **skipping** — because `template_enabled: false`.

13

Real run

```
# localhost with sudo prompt
$ ansible-playbook site.yml -i inventory.ini --limit localhost --ask-become-pass -v

# as root in WSL (no password)
$ sudo ansible-playbook site.yml -i inventory.ini --limit localhost -v
```

[**PROOF · Run it TWICE**] The second run must report **ok** everywhere — **idempotency proof**. First run: **changed** (things installed) · second run: **ok** (nothing left to do). If the second run still says changed, something in the tasks is non-idempotent — fix before continuing.

[The three-gate testing ladder]

① **syntax-check** parses ✓② **--check --diff** predicts, touches nothing③ **real run x2** applies, then proves idempotent

[**DEBUG · Why this ladder works**] Each gate catches a different class of failure: syntax errors cost seconds, prediction errors cost a dry run, idempotency violations cost one extra real run. Reaching the real run with all three green means the blast radius of any remaining bug is nearly zero.

🔑 **EXAM** The gates in order: `--syntax-check` → `--check --diff` (expect changed + skipped-for-template_enabled) → real run **twice** (changed → ok). And `--limit localhost` keeps the first tests off the remote web hosts.

▶▶ The play is green — now prove nginx actually serves: → **Steps 14-15**



Verify nginx • Prove Reusability

14 Verify nginx — six ways

```
$ service nginx status      # must be active (running)
$ ss -tlnp | grep 80       # port 80 listening
$ nginx -t                 # config test ok
$ curl -s http://localhost/ | head -20 # styled index page
$ cat /var/www/html/index.html
$ curl -s http://127.0.0.1/ -H "Host: eslam.com" | head # vhost test
```

15 Prove reusability — a second package, zero role edits

```
- role: generic_package
vars:
  package_name: redis-server
  service_name: redis-server
  template_enabled: false
```

Same role, different vars — redis installs. The role code was **never touched**. That's Objective 2 closed.

[Objective 4 – what green looks like]

service: active (running)

ss: port 80 LISTEN

nginx -t: ok

curl: styled page ✓

[KEY IDEA] Step 14 is deliberately six checks, not one: **service alive, port bound, config valid, content served, file landed, vhost answers**. Each proves a different layer — systemd, network, syntax, rendering, file placement, name-based routing. One green curl alone could hide a broken config.

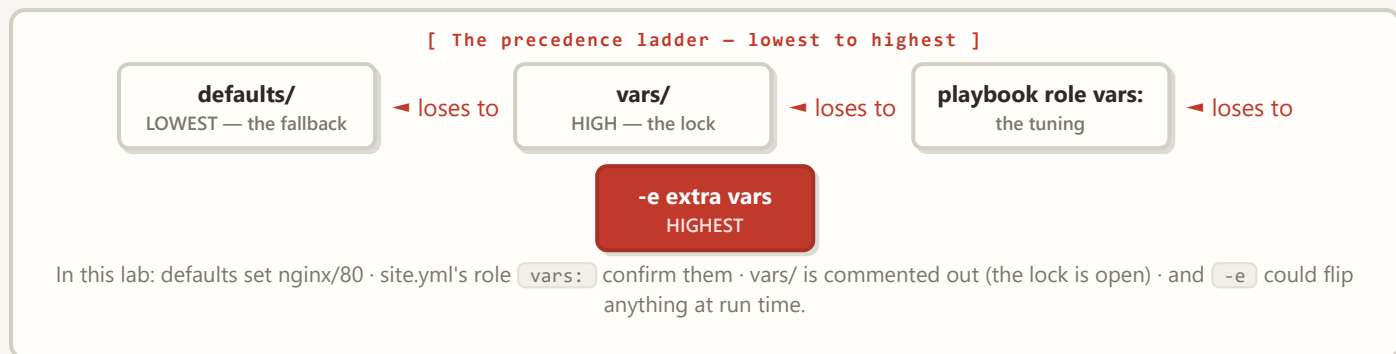
[DEBUG • The vhost test] `-H "Host: eslam.com"` fakes the browser's Host header without DNS. It tests name-based routing locally — the answer should differ from the default site (see the eslam.com row in Troubleshooting if it doesn't).

🎓 EXAM Verification set: `service status` · `ss -tlnp | grep 80` · `nginx -t` · `curl localhost` · `cat index.html` · `curl -H Host:` — then the reuse proof: **redis-server via the same role, vars only**.

🔗 Built and proven — now the *why* behind the variable magic: → [Precedence & Anatomy](#)

Variable Precedence + Role Anatomy

Objective 3's trickiest concept: **who wins when the same variable is set in several places?** Ansible's answer is a precedence ladder — and this role uses every rung of it.



A The role's anatomy — folder by folder, in this project

Folder	In this task
tasks/	Cache update (Debian-only) · install via <code>package</code> · service guard · template/index deploys
handlers/	Restart + Reload — once at end of play, only if notified, both <code>ignore_errors</code>
templates/	app.conf.j2 (nginx if/fork) · index.html.j2 (styled page) · nginx.conf.j2 (distro variant)
defaults/	The 8 tunables — including <code>service_name</code> referencing <code>package_name</code>
vars/	The high-precedence lock — commented out in this lab
meta/	galaxy_info: author, MIT license, Ubuntu/Debian platforms, tags · dependencies: []

► TASK [The experiment you can run] see precedence yourself

🔬 Test 1 — playbook beats defaults

site.yml sets `app_port: 80` over defaults' `80`. Change the playbook to `app_port: 8080` → the template renders 8080. Defaults only apply when **nobody else spoke**.

🔬 Test 2 — vars/ beats the playbook

Uncomment `package_name: nginx` in vars/, set the playbook to redis → the play still installs **nginx**. The lock wins. Re-comment to restore.

[WATCH OUT] The ladder is why the vars/ lock must stay commented in this lab — with it active, `site.yml`'s vars are dead letters and the "reusability" demo can't work. High precedence isn't "more important," it's **"wins the fight."**

🎓 EXAM The ladder by heart: **defaults/ < vars/ < playbook role vars: < -e extra vars** — lowest to highest. And the design rule that follows: put tunables in **defaults/**, put invariants in **vars/**, and let callers tune through playbook vars or `-e`.

🔗 Every concept this task exercises, mapped: → [The Concepts in Play](#)

The Ansible Concepts — All in Play

Objective 3 requires every concept "used correctly." Here's the map: each concept, where it lives in this project, and the rule it demonstrates.

Concept	Where it lives here	The rule it proves
Variables priority	defaults/main.yml · vars/main.yml · site.yml <code>vars: {}</code> · potential <code>-e</code>	defaults lowest → -e highest; vars/ = the lock (page 11)
Handlers	handlers/main.yml — Restart + Reload, notified by the install task	Run once at end of play , only if notified · <code>ignore_errors</code> so one bad service doesn't abort
Templates (Jinja2)	app.conf.j2 (if/fork by package) · index.html.j2 (styled page) · nginx.conf.j2	<code>{{ var }}</code> holes + <code>{% if %}</code> logic + <code> default(1024)</code> fallbacks
Idempotency	The whole run — proven in Step 13	Run twice → second reports <code>ok</code> , not <code>changed</code>
Conditionals	<code>when: ansible_os_family == "Debian"</code> · <code>when: service_name is defined...</code> · <code>template_enabled bool</code>	Guards make one role behave differently per OS, package, and flag
Loose coupling	Role code contains zero package names	Everything arrives via variables (Objective 2)
Role convention	The standard 7-folder tree + meta	Any engineer navigates it without a guide

[[DEBUG](#) · [Reading the task back](#)] The task spec says "all of them used correctly" — the table is your self-audit. For each row, open the named file and point at the line. If you can't, that concept isn't actually in your project yet.

[[WATCH OUT](#) · [Two subtle misuses](#)] **1.** Using `apt` instead of `package` in the install task — works on Ubuntu, breaks the "generic" claim. **2.** Forgetting `| bool` on `template_enabled` — YAML `false` vs string `"false"` silently flips the condition.

🎓 **EXAM** Concept-to-file mapping in one breath: **precedence** → defaults/vars/site/-e · **handlers** → Restart+Reload with `ignore_errors` · **templates** → app.conf.j2 fork + index.html.j2 · **idempotency** → the twice-run proof · **conditionals** → when/ansible_os_family + service_name guard + `template_enabled | bool`.

▶▶ What the grader sees when everything is green: → [The Expected Result](#)

✓ The Expected Result — Proven

"Verified with the full command loop" — here is what each of the four gates looks like when it's green:

✓ 1 · syntax-check passes

```
$ ansible-playbook site.yml -i inventory.ini --syntax-check
playbook: site.yml
```

No errors — the YAML and structure parse cleanly.

✓ 2 · dry-run shows expected counts

```
$ ansible-playbook ... --check --diff -v
changed: install / index tasks
skipping: config template
(template_enabled: false)
```

Predicts changes without touching the system.

✓ 3 · real run installs + starts

```
$ ansible-playbook site.yml ...
changed: Install package nginx
changed: Ensure service nginx...
PLAY RECAP — second run:
ok=N changed=0 failed=0
```

First run applies; second run proves idempotent.

✓ 4 · curl returns the styled page

```
$ curl -s http://localhost/ | head -20
<h1>...Deployed via generic_package...</h1>
package · host · service · port shown
```

The rendered index.html,j2, served live by nginx.

► TASK [The full green board] all objectives at once

01 structure ✓

02 generic ✓

03 concepts ✓

04 nginx proof ✓

05 verified ✓

Five green chips = task complete. Any red → the Troubleshooting table (page 15) is ordered by likelihood.

[KEY IDEA] Notice what the four checks prove **together**: syntax (it parses), prediction (it would do the right things), application (it did them), and rendering (the result is visible over HTTP). Each gate covers a failure class the previous one can't see — that's why the loop is "full."

[ASIDE · The badge] The styled page carries a pulsing "**Service Active & Enabled**" badge — pure CSS gradient and animation. A human glance at the browser is a sixth verification layer: if the pulse is beating, nginx is up and the template rendered *and* is being served.

🎓 EXAM Memorize the four-gate loop: **syntax** → **dry** → **real** → **curl**. Asked "how do you verify an Ansible deployment?", that order — parse, predict, apply, observe — is the professional answer.

▶ Before calling it done, tick every box: → **Completion Checklist**



Completion Checklist

The task's own completion gate. Tick every box — a single unchecked one means the task isn't finished, no matter how green the runs look.



Syntax check

`ansible-playbook --syntax-check` returns `playbook: site.yml` without errors.



Idempotent dry + real run

Dry run `--check --diff` shows `changed` on first run, `ok` on second — the idempotency proof.



Service is live

`service nginx status` shows `active (running)`.



The styled page serves

`curl http://localhost/` returns the styled page from `index.html.j2`.



Reusability proven

Same role installs a second package (redis, apache2...) with **only var changes**.



Everything explained and committed

`ansible.cfg`, `inventory.ini`, `site.yml`, `roles/generic_package/*` — all explained and committed to **git**.

[**KEY IDEA**] Read the six boxes as the task's **definition of done** — structure, idempotency, service, content, reuse, version control. The last box matters most and fails most often: a working project that isn't in git doesn't satisfy the spec.

[**DEBUG · Suggested commit**] `git add -A && git commit -m "generic_package role: install+configure any package via vars; nginx deploy verified; idempotency proven"` — one commit that says what the grader needs to read.



EXAM The six-gate gate: **syntax ✓ idempotency ✓ service ✓ content ✓ reuse ✓ git ✓**. All six, in order, is "complete."

🔗 If any box won't tick — the rescue table: → [Troubleshooting](#)

Troubleshooting — Symptom → Cause → Fix

Five failures this task actually produces, each with its root cause and exact fix — ordered by how likely you are to hit them:

Symptom	Cause	Fix
<code>config file = None</code> + "world writable directory" warning	<code>/mnt/c</code> NTFS mounted 777	Copy project to <code>~/ansible</code> , <code>chmod 644 ansible.cfg</code> , run with <code>ANSIBLE_CONFIG=~/ansible/ansible.cfg</code>
<code>sudo: a password is required</code>	<code>become</code> needs sudo	Add <code>--ask-become-pass</code> — or run <code>wsl --user root / sudo</code>
<code>curl</code> still shows the old eslam.com page	eslam.com vhost is the default site	Role already writes <code>/var/www/eslam/index.html</code> ; enable default site: <code>sudo ln -s /etc/nginx/sites-available/default /etc/nginx/sites-enabled/</code>
Handler "Restart service" fails on a non-service package	That package has no service	<code>ignore_errors: true</code> on handlers (already set) — or set <code>service_name: ""</code> to skip the service task entirely
Role not found	Wrong <code>roles_path</code>	Confirm <code>roles_path = roles</code> in <code>ansible.cfg</code> — and that the role directory name matches the playbook reference

[WATCH OUT · The #1 trap, expanded] The `/mnt/c` issue is environmental, not code: OneDrive + NTFS makes everything 777, and Ansible **refuses to trust it** for config discovery. The copy-to-`~/ansible` fix is a one-time move — then `ANSIBLE_CONFIG` pins the path so there's no ambiguity about which `ansible.cfg` is loading.

[DEBUG · Reading the table like an engineer] The pattern across all five rows: the **symptom** is always one layer below the **cause**. "Role not found" is a config problem; "password required" is an environment problem; the `curl` echo is a routing problem. Diagnose the layer, then apply the row's fix — don't chase the error text alone.

🔑 **TAKEAWAY** The five-row rescue kit: NTFS/mnt-c config refusal · missing become password · eslam.com default-site echo · non-service package restart failure · `roles_path` mismatch. Symptom → cause → fix, each in one line.

[SHIP IT] Syntax ✓ · dry run ✓ · two real runs ✓ · six nginx verifications ✓ · redis via the same role ✓ · six checklist boxes ticked ✓ — **the task is complete, and the role is genuinely reusable.** 🎉